

SQL Server AlwaysOn Availability Groups - Fundamentals

Primary & Secondary Replicas — What Every DBA *Must* Understand

Always On Availability Groups (AGs) are the backbone of **High Availability (HA)** and **Disaster Recovery (DR)** in modern SQL Server environments.

But real success doesn't come from *how many* replicas you have — it comes from **how you design and use them**.

Let's break it down with **real DBA scenarios and hands-on examples**.

◆ Primary Replica — The Write Engine

The **Primary Replica** is the heart of the Availability Group.

What it does

- ✓ Handles **all read-write workloads**
- ✓ Generates transaction log records
- ✓ Sends log blocks to secondary replicas
- ✓ Owns the AG at any given time

💡 **Rule:** Only one primary exists per Availability Group.

Real-Time Scenario (AdventureWorks2019)

You're running an **e-commerce workload** using AdventureWorks2019.

```
USE AdventureWorks2019;
GO
INSERT INTO Sales.SalesOrderHeader
(
    OrderDate, DueDate, ShipDate, Status,
    OnlineOrderFlag, CustomerID, BillToAddressID,
    ShipToAddressID, ShipMethodID, SubTotal, TaxAmt, Freight
)
VALUES
(
    GETDATE(), GETDATE()+7, GETDATE()+2, 1,
    1, 11000, 921, 921, 5, 1000, 80, 25
);
```

This transaction:

- Executes **only on the Primary Replica**
- Generates log records
- Ships them to all secondary replicas

If the primary goes down, a **secondary can take over**, but only after synchronization rules are met.

◆ Secondary Replicas — Scale, Safety & Recovery

Secondary replicas are not just passive copies — they are **strategic tools**.

What they can do

- ✓ Apply transaction logs from the primary
- ✓ Serve **read-only workloads**
- ✓ Run **backups**
- ✓ Act as **failover targets**

Scenario 1: Read-Only Reporting Replica

Your reporting team runs heavy queries that slow down production.

🔑 Solution: Offload reporting to a **readable secondary replica**.

SELECT

SalesOrderID,
OrderDate,
TotalDue

FROM Sales.SalesOrderHeader

WHERE OrderDate >= '2024-01-01';

📌 Executed on:

- Secondary replica
- Read-only mode
- Zero impact on production writes

Scenario 2: Backup on Secondary

Instead of backing up on the primary:

BACKUP DATABASE AdventureWorks2019

TO DISK = 'E:\SQLBackups\AW2019_Full.bak'

WITH COPY_ONLY;

📌 Benefits:

- Reduces IO and CPU pressure on primary
- Faster user response times
- Best practice in AG environments



Maximum Replicas — What SQL Server Allows

SQL Server 2019 → 2025 Limits



Up to 9 replicas total

- 1 Primary
- Up to 8 Secondary replicas



Up to 3 replicas can be:

- **Synchronous commit**
- **Automatic failover enabled**



Remaining replicas must be:

- **Asynchronous commit**
- Typically used for DR or reporting



Real-World Architecture Example



Global Enterprise Setup

Location	Replica Type	Commit Mode	Purpose
Data Center 1	Primary	Sync	Production OLTP
Data Center 1	Secondary	Sync	Auto Failover
Data Center 2	Secondary	Sync	HA
Data Center 3	Secondary	Async	DR
Cloud (Azure VM)	Secondary	Async	Reporting + DR



Why this works

- HA protected locally with synchronous replicas
- DR protected remotely with async replicas

- Reporting offloaded safely

Failover Example (AdventureWorks2019)

ALTER AVAILABILITY GROUP AG_AdventureWorks

FAILOVER;

 What happens:

- One synchronous secondary becomes primary
- No data loss
- Applications reconnect automatically (with proper listener setup)

DBA Takeaways (Real-World Wisdom)

- ✓ More replicas ≠ better performance
- ✓ Synchronous replicas increase **latency**
- ✓ Asynchronous replicas protect against **regional failures**
- ✓ Readable secondaries reduce reporting load
- ✓ Backup on secondaries is a **must**

Final Thought

Always On Availability Groups are not just HA — they are a platform for scalability, resilience, and smart workload distribution.

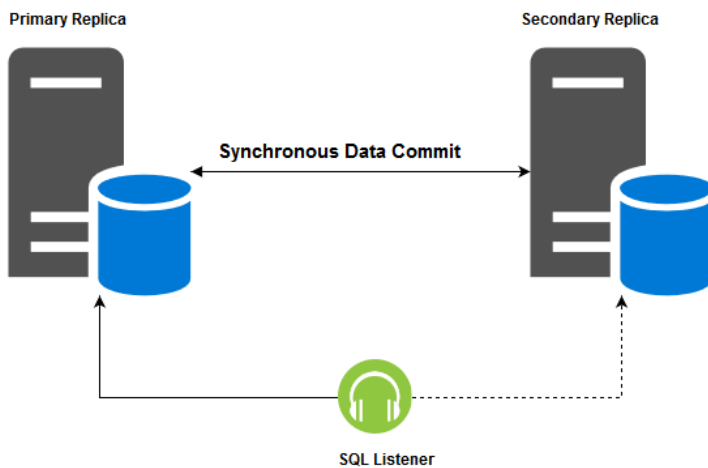
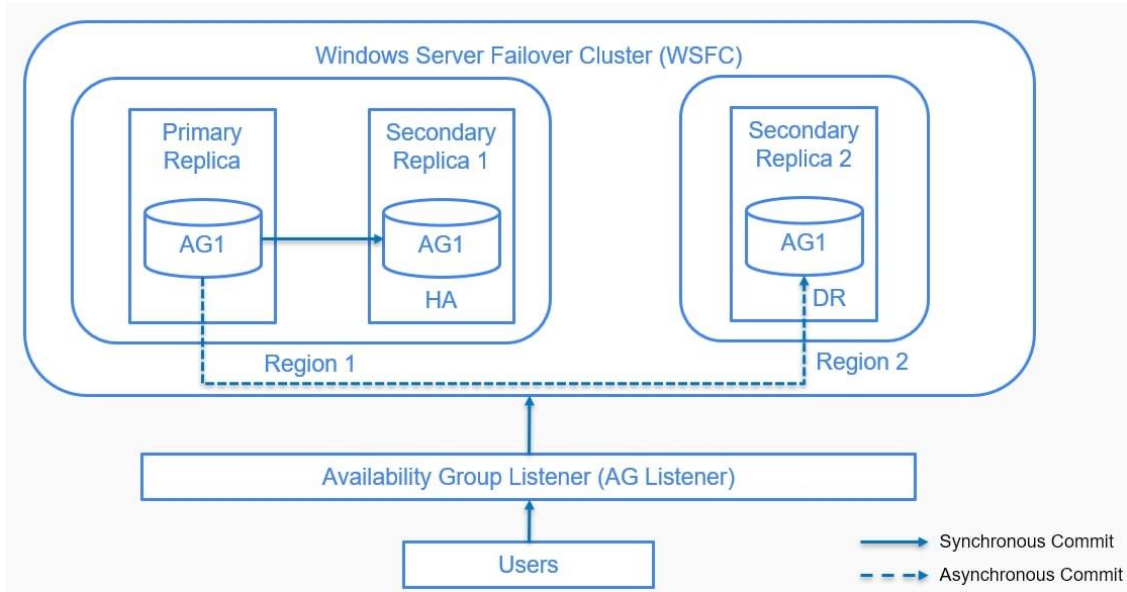
The real DBA skill lies in:

Choosing the right number of replicas, in the right mode, for the right workload.

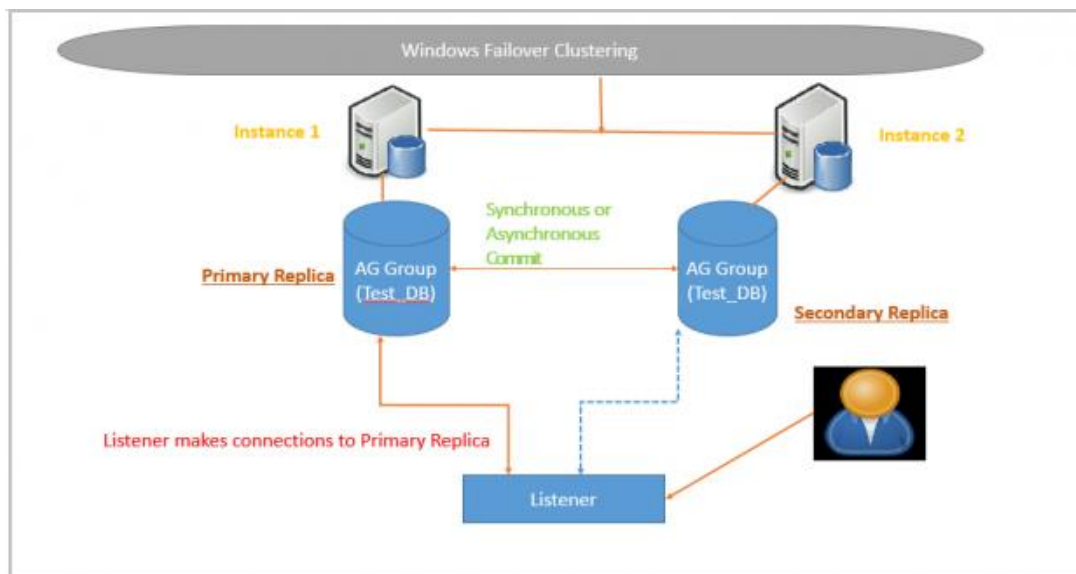
<https://www.sqldbachamps.com/>

Complete architecture, monitoring, and hands-on setup using AdventureWorks2019 on SQL Server 2019–2025.

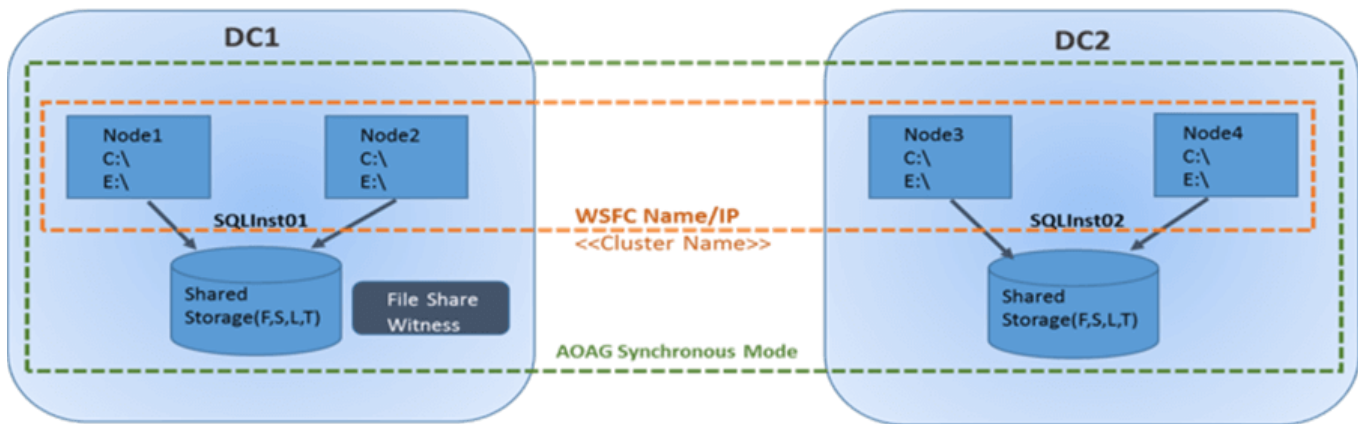
Sample Always On Availability Group Architecture



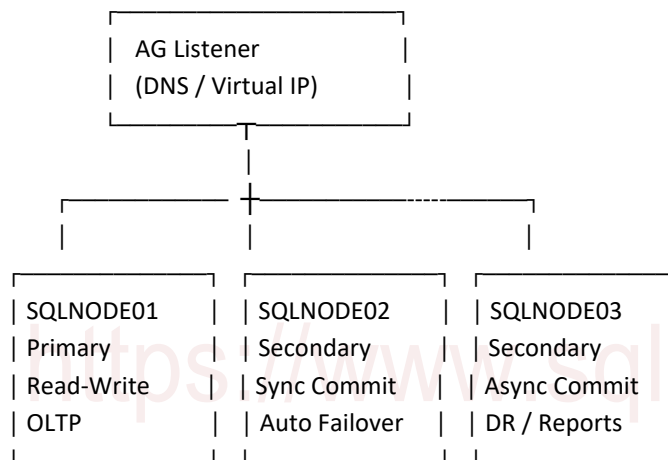
achamps.com/



1. HA – FCIs with Shared Storage in each DC. Both FCIs nodes are part of single WSFC.
2. DR- AlwaysON Availability Group between both FCIs.



Reference Architecture (Real-World Design)



Design Intent

Replica	Commit Mode	Failover	Usage
SQLNODE01	Synchronous	—	Production
SQLNODE02	Synchronous	Automatic	HA
SQLNODE03	Asynchronous	Manual	DR / Reporting

Always On AG Monitoring Queries (DBA Toolkit)

1) Replica Health & Sync Status

SELECT

```

ar.replica_server_name,
rs.role_desc,
rs.connected_state_desc,
rs.synchronization_state_desc,
rs.synchronization_health_desc

```

```

FROM sys.dm_hadr_availability_replica_states rs
JOIN sys.availability_replicas ar
ON rs.replica_id = ar.replica_id;

```

2) Database-Level Sync Lag

SELECT

```
db_name(drs.database_id) AS DatabaseName,  
drs.synchronization_state_desc,  
drs.log_send_queue_size,  
drs.redo_queue_size,  
drs.last_commit_time
```

FROM sys.dm_hadr_database_replica_states drs;



Watch out for:

- Large log_send_queue_size
- Growing redo_queue_size

3) Readable Secondary Routing Check

SELECT

```
ag.name AS AGName,  
ar.replica_server_name,  
ar.secondary_role_allow_connections_desc AS ReadableSecondary
```

FROM sys.availability_groups ag

JOIN sys.availability_replicas ar

```
ON ag.group_id = ar.group_id;
```

4) Backup Preference Verification

SELECT

```
ag.name,  
ag.automated_backup_preference_desc
```

FROM sys.availability_groups ag;

5) Failover Readiness (Critical DBA Check)

SELECT

```
ar.replica_server_name,  
ar.failover_mode_desc,  
ar.availability_mode_desc
```

FROM sys.availability_replicas ar;

Step-by-Step: AdventureWorks2019 Always On AG Setup



Prerequisites (Must-Have)

- ✓ Windows Server Failover Cluster (WSFC)
- ✓ SQL Server 2019–2025 Enterprise Edition
- ✓ Full Recovery Model
- ✓ Same SQL patch level
- ✓ Always On enabled in SQL Server Configuration Manager

Step 1) Enable Always On

SQL Server Configuration Manager

SQL Server Services → SQL Server → Properties

✓ Enable Always On Availability Groups

Restart SQL Service

Step 2) Prepare AdventureWorks2019

```
ALTER DATABASE AdventureWorks2019 SET RECOVERY FULL;  
GO  
BACKUP DATABASE AdventureWorks2019  
TO DISK = 'E:\Backup\AW2019_Full.bak';
```

```
BACKUP LOG AdventureWorks2019  
TO DISK = 'E:\Backup\AW2019_Log.trn';
```

Restore on secondary replicas **WITH NORECOVERY**.

Step 3) Create Availability Group

```
CREATE AVAILABILITY GROUP AG_AdventureWorks  
FOR DATABASE AdventureWorks2019  
REPLICA ON  
'SQLNODE01' WITH
```

```
(  
    ENDPOINT_URL = 'TCP://SQLNODE01:5022',  
    AVAILABILITY_MODE = SYNCHRONOUS_COMMIT,  
    FAILOVER_MODE = AUTOMATIC,  
    SEEDING_MODE = AUTOMATIC
```

```
),  
'SQLNODE02' WITH
```

```
(  
    ENDPOINT_URL = 'TCP://SQLNODE02:5022',  
    AVAILABILITY_MODE = SYNCHRONOUS_COMMIT,  
    FAILOVER_MODE = AUTOMATIC,  
    SEEDING_MODE = AUTOMATIC
```

```
),  
'SQLNODE03' WITH
```

```
(  
    ENDPOINT_URL = 'TCP://SQLNODE03:5022',  
    AVAILABILITY_MODE = ASYNCHRONOUS_COMMIT,  
    FAILOVER_MODE = MANUAL,  
    SEEDING_MODE = AUTOMATIC
```

```
);
```

Step 4) Join Secondaries

```
ALTER AVAILABILITY GROUP AG_AdventureWorks JOIN;  
GO
```

```
ALTER DATABASE AdventureWorks2019 SET HADR AVAILABILITY GROUP = AG_AdventureWorks;
```

Step 5) Configure Listener

```
ALTER AVAILABILITY GROUP AG_AdventureWorks  
ADD LISTENER 'AGListener'
```

```
(  
    WITH IP  
    (  
        ('10.10.10.100','255.255.255.0')
```

```
),  
PORT = 1433
```

```
);
```

📌 Applications connect using:
AGListener,1433

Step 6) Test Failover

```
ALTER AVAILABILITY GROUP AG_AdventureWorks FAILOVER;
```

- ✓ No data loss
- ✓ Minimal downtime
- ✓ Transparent to applications

🔗 DBA Best Practices (From Production)

- ✓ Keep ≤ 3 synchronous replicas
- ✓ Use async for DR only
- ✓ Offload reporting & backups
- ✓ Monitor log send & redo queues
- ✓ Test failover quarterly

🧠 Final Note

An Availability Group is not “set and forget”.

A **well-monitored AG** is the difference between **five nines uptime** and a **2 AM outage call** 📞.

<https://www.sqldbachamps.com/>

Senior-DBA-grade guide covering **real-world troubleshooting, performance tuning, Azure hybrid design, and interview prep** for SQL Server Always On Availability Groups (SQL Server 2019–2025).

Always On AG Troubleshooting Playbook

1. Replica Not Synchronizing

Symptoms

- NOT SYNCHRONIZED
- Growing redo queue
- Delayed failover readiness

SELECT

```
ar.replica_server_name,
drs.synchronization_state_desc,
drs.log_send_queue_size,
drs.redo_queue_size
```

```
FROM sys.dm_hadr_database_replica_states drs
JOIN sys.availability_replicas ar
ON drs.replica_id = ar.replica_id;
```

Root Causes

Cause	Fix
Network latency	Validate NIC speed, MTU, packet loss
Disk IO bottleneck	Move logs/data to faster storage
CPU pressure	Reduce sync replicas, fix waits

2. Automatic Failover Not Working

SELECT

```
replica_server_name,
availability_mode_desc,
failover_mode_desc
```

```
FROM sys.availability_replicas;
```

Must-Haves

- ✓ Synchronous commit
- ✓ Automatic failover
- ✓ Healthy synchronization
- ✗ Async replicas **cannot** auto-failover

3. Log Send Queue Growing Rapidly

SELECT

```
db_name(database_id) AS DB,
log_send_queue_size
```

```
FROM sys.dm_hadr_database_replica_states;
```

DBA Fixes

- Compress network traffic
- Remove unused sync replicas
- Reduce transaction size

Latency Tuning for Synchronous Replicas

Why Latency Happens

Every commit waits for **log hardening on secondary**.

Tuning Checklist

1) Reduce Number of Sync Replicas

✓ Max 2–3 synchronous replicas

```
ALTER AVAILABILITY GROUP AG_AdventureWorks
MODIFY REPLICA ON 'SQLNODE03'
WITH (AVAILABILITY_MODE = ASYNCHRONOUS_COMMIT);
```

2) Optimize Network

- ✓ Same subnet if possible
- ✓ 10–25 Gbps NIC
- ✓ Low-latency switches

🚀 Target RTT: < 5 ms

3) Optimize Log IO

Component	Best Practice
Log disk	Dedicated NVMe / Premium SSD
TempDB	Separate storage
Antivirus	Exclude .ldf

4) Monitor Commit Latency

SELECT

```
ar.replica_server_name,
db_name(drs.database_id) AS database_name,
drs.last_commit_time
```

FROM sys.dm_hadr_database_replica_states drs

JOIN sys.availability_replicas ar

```
ON drs.replica_id = ar.replica_id
```

ORDER BY drs.last_commit_time DESC;

☁ Azure Hybrid Always On AG Design

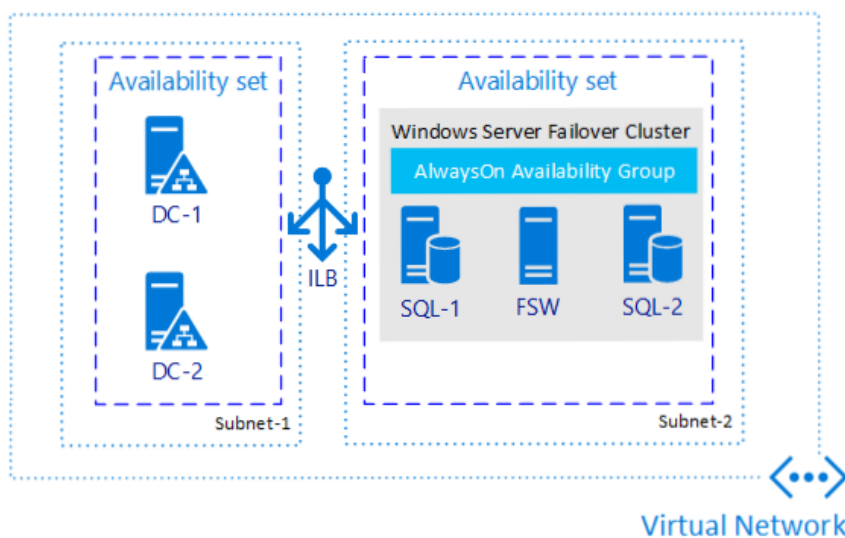


Image Reference:

<https://learn.microsoft.com/en-us/azure/azure-sql/virtual-machines/windows/availability-group-overview?view=azuresql>

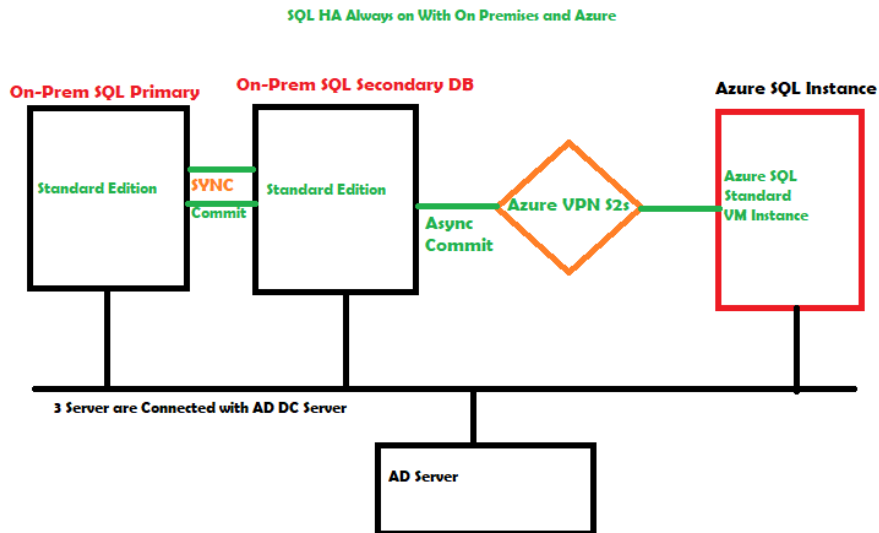


Image Reference:

<https://learn.microsoft.com/en-us/answers/questions/1473685/on-premises-and-azure-sql-vm-always-on-availabilit>

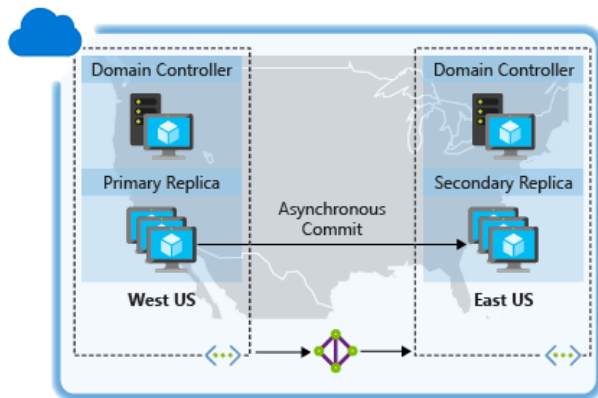
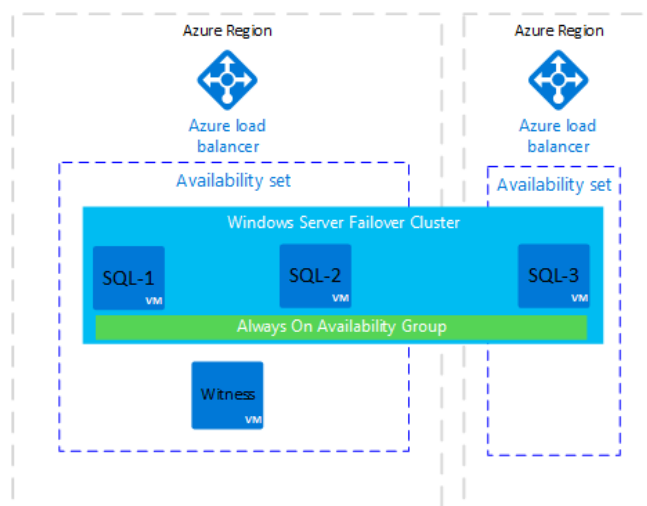


Image Reference:

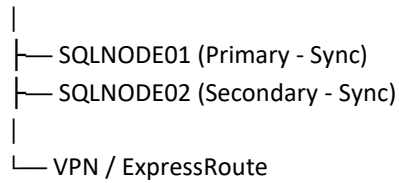
<https://learn.microsoft.com/en-us/azure/azure-sql/virtual-machines/windows/business-continuity-high-availability-disaster-recovery-hadr-overview?view=azuresql>



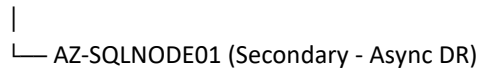
Reference: <https://learn.microsoft.com/en-us/azure/azure-sql/virtual-machines/windows/availability-group-manually-configure-multiple-regions?view=azuresql>

Hybrid Architecture (Most Common)

On-Premises Datacenter



Azure Virtual Network



Azure Hybrid Benefits

- ✓ Offsite DR without new datacenter
- ✓ Pay-as-you-go compute
- ✓ Azure Backup + Azure Monitor
- ✓ Easy scale-out for reporting

Azure-Specific Settings

ALTER AVAILABILITY GROUP AG_AdventureWorks

MODIFY REPLICA ON 'AZ-SQLNODE01'

WITH (

AVAILABILITY_MODE = ASYNCHRONOUS_COMMIT,

FAILOVER_MODE = MANUAL

);

 **Never use synchronous commit over VPN**

Interview-Ready Always On AG Questions & Answers

Q1) Difference between Failover Cluster and Availability Group?

Answer:

WSFC protects instances; AGs protect databases and allow read replicas.

Q2) How many replicas support automatic failover?

Answer:

Maximum 3 (1 primary + 2 synchronous secondaries).

Q3) Can AG work without a listener?

Answer:

Yes, but applications must manage replica connection logic manually.

Q4) Why does synchronous commit increase latency?

Answer:

Transactions wait for secondary log hardening before commit acknowledgment.

Q5) Best practice for backups in AG?

Answer:

Run backups on **secondary replicas** using backup preferences.

Q6) How do you monitor AG health?

Answer:

DMVs:

- sys.dm_hadr_database_replica_states
- sys.dm_hadr_availability_replica_states

Q7) Async vs Sync — when to use?

Mode	Use Case
Sync	HA, zero data loss
Async	DR, long-distance

Final DBA Wisdom

Always On Availability Groups are not about replicas — they are about intelligent workload placement.

- ✓ Fewer sync replicas = lower latency
- ✓ Async replicas = DR safety net
- ✓ Azure = modern, cost-effective DR
- ✓ Monitoring = survival skill

Complete production DBA scripts we can run, reuse.

Everything works on **SQL Server 2019 → 2025** and is safe for production diagnostics.

⚡ 1) AG Latency Calculation Script (Near-Real Time)

Goal: Estimate commit lag by comparing primary vs secondary commit times.

WITH CommitLag AS

```
(
    SELECT
        ar.replica_server_name,
        db_name(drs.database_id) AS database_name,
        drs.last_commit_time,
        LAG(drs.last_commit_time) OVER
        (
            PARTITION BY drs.database_id
            ORDER BY drs.last_commit_time
        ) AS previous_commit_time
    FROM sys.dm_hadr_database_replica_states drs
    JOIN sys.availability_replicas ar
        ON drs.replica_id = ar.replica_id
)
SELECT
    replica_server_name,
    database_name,
    DATEDIFF(MILLISECOND, previous_commit_time, last_commit_time) AS commit_latency_ms
FROM CommitLag
WHERE previous_commit_time IS NOT NULL
ORDER BY commit_latency_ms DESC;
```

🔍 Interpretation

Value	Meaning
< 10 ms	Excellent
10–50 ms	Acceptable
> 100 ms	Investigate immediately

🕒 2) Detect Commit Delay in Milliseconds (Primary vs Secondary)

Most accurate way DBAs detect sync pain

```
SELECT
    ar.replica_server_name,
    db_name(drs.database_id) AS database_name,
    drs.log_send_queue_size, -- KB
    drs.redo_queue_size,    -- KB
    DATEDIFF
    (
        MILLISECOND,
        drs.last_hardened_time,
        drs.last_commit_time
    ) AS commit_delay_ms
FROM sys.dm_hadr_database_replica_states drs
JOIN sys.availability_replicas ar
    ON drs.replica_id = ar.replica_id
```

ORDER BY commit_delay_ms DESC;

Red flags

- commit_delay_ms > 50
- Log queue growing faster than redo queue

3) Single AG Health Dashboard Query (All-in-One)

Bookmark this — DBAs use it daily

SELECT

```
ag.name AS AG_Name,
ar.replica_server_name,
rs.role_desc,
rs.connected_state_desc,
rs.synchronization_health_desc,
drs.synchronization_state_desc,
drs.log_send_queue_size AS log_send_kb,
drs.redo_queue_size AS redo_kb,
drs.last_commit_time
```

FROM sys.availability_groups ag

JOIN sys.availability_replicas ar

ON ag.group_id = ar.group_id

JOIN sys.dm_hadr_availability_replica_states rs

ON ar.replica_id = rs.replica_id

JOIN sys.dm_hadr_database_replica_states drs

ON ar.replica_id = drs.replica_id

ORDER BY log_send_kb DESC;

✓ Replica health

✓ Sync status

✓ Latency indicators

✓ DR readiness

4) Why log_send_queue Grows Even When CPU Is Low

Root Causes (Very Common)

Cause	Explanation
Network latency	Log blocks wait for ACK
Slow log disk on secondary	Redo can't keep up
Large transactions	Log blocks held longer
Synchronous commit	Primary waits on secondary
Antivirus on .ldf	IO delay (huge offender)

Fix Order

1. Check **network RTT**
2. Check **secondary log disk latency**
3. Reduce sync replicas
4. Break large transactions
5. Exclude SQL paths from AV

5) AG Disaster Recovery Drill (Runbook)

Quarterly DR Test (Zero Guesswork)

Step 1 — Validate Readiness

SELECT

```
ar.replica_server_name,  
ars.synchronization_health_desc
```

FROM sys.dm_hadr_availability_replica_states AS ars

JOIN sys.availability_replicas AS ar

```
ON ars.replica_id = ar.replica_id;
```

✓ Must be **HEALTHY**

Step 2 — Simulate Failure

ALTER AVAILABILITY GROUP AG_AdventureWorks FAILOVER;

Step 3 — Application Validation

- Connect using **Listener**
- Confirm read-write access
- Validate row counts

Step 4 — Failback

ALTER AVAILABILITY GROUP AG_AdventureWorks FAILOVER;

 Document RTO + RPO every test

6) Real Production Failure Case Study

Scenario

- 2 sync replicas
- 1 async DR
- Sudden OLTP slowdown

Findings

Metric	Value
Log send queue	25 GB
Network RTT	18 ms
CPU	30%

Root Cause

→ Synchronous commit across **different datacenters**

Fix

```
ALTER AVAILABILITY GROUP AG_Prod  
MODIFY REPLICA ON 'DRNODE'  
WITH (AVAILABILITY_MODE = ASYNCHRONOUS_COMMIT);
```

Result

- ✓ TPS restored
- ✓ Commit latency dropped from 120 ms → 8 ms

☁ 7) AG vs Azure SQL Managed Instance (MI)

Feature	Always On AG	Azure SQL MI
OS control	Full	None
Read replicas	Yes	Yes
Cross-region DR	Manual	Built-in
Patching	DBA-managed	Automatic
Cost predictability	Hardware-based	Consumption-based
Best for	Enterprise control	Cloud-first apps

📌 Rule of Thumb

- Need OS & SQL control → **AG**
- Want managed HA → **Azure MI**

🧠 8) Printable AG Interview Cheat Sheet

Key Facts

- ✓ Max replicas: **9**
- ✓ Auto-failover: **3 max**
- ✓ Sync = HA
- ✓ Async = DR

Common Questions

Q: Can async replica auto-failover?

A: ❌ No

Q: Best backup location?

A: Secondary replica

Q: Why sync commit hurts performance?

A: Log hardening wait

Q: Listener purpose?

A: Abstracts replica connections

Golden Rules

- ✓ Keep sync replicas close
- ✓ Monitor queues, not CPU
- ✓ Test failover quarterly
- ✓ Never sync over WAN

🎯 Final DBA Truth

Always On Availability Groups don't fail — designs do.

Automated Alerting Scripts for AlwaysOn AGs

These scripts are designed to run via **SQL Agent Jobs** (every 1–5 minutes) and **raise alerts only when action is needed**.

1) Alert: AG Replica Not Healthy

Condition

- Replica disconnected
- Sync health NOT healthy

IF EXISTS

```
(
  SELECT 1
  FROM sys.dm_hadr_availability_replica_states
  WHERE synchronization_health_desc <> 'HEALTHY'
     OR connected_state_desc <> 'CONNECTED'
)
BEGIN
  RAISERROR
  (
    'AG ALERT: One or more replicas are unhealthy or disconnected.',
    16, 1
  );
END
```

Action:

Attach this job to:

- SQL Agent Alert
- Email Operator
- PagerDuty / webhook (if integrated)

2) Alert: Log Send Queue Threshold (Latency Risk)

Condition

- Log send queue > 5 GB (adjust for your workload)

IF EXISTS

```
(
  SELECT 1
  FROM sys.dm_hadr_database_replica_states
  WHERE log_send_queue_size > 5120000 -- KB (5 GB)
)
BEGIN
  RAISERROR
  (
    'AG ALERT: Log send queue exceeds safe threshold.',
    16, 1
  );
END
```

Why this matters:

Growing log queue = network, disk, or sync commit pressure.

3) Alert: Redo Queue Backlog (Secondary Falling Behind)

IF EXISTS

```
(
  SELECT 1
  FROM sys.dm_hadr_database_replica_states
  WHERE redo_queue_size > 5120000 -- KB
)
BEGIN
  RAISERROR
  (
    'AG ALERT: Redo queue backlog detected on secondary.',
    16, 1
  );
END
```

Typical causes

- Slow disk on secondary
- CPU starvation
- Antivirus scanning MDF/LDF

4) Alert: Commit Delay (Synchronous Pain)

Condition

- Commit delay > 100 ms

IF EXISTS

```
(
  SELECT 1
  FROM sys.dm_hadr_database_replica_states
  WHERE DATEDIFF
  (
    MILLisecond,
    last_hardened_time,
    last_commit_time
  ) > 100
)
BEGIN
  RAISERROR
  (
    'AG ALERT: Commit latency exceeds 100 ms.',
    16, 1
  );
END
```

Use case:

Immediate detection of sync-replica performance regression.

5) Alert: Automatic Failover Not Possible

IF EXISTS

```
(
  SELECT 1
```

```
FROM sys.availability_replicas
WHERE availability_mode_desc = 'SYNCHRONOUS_COMMIT'
AND failover_mode_desc <> 'AUTOMATIC'
)
BEGIN
    RAISERROR
    (
        'AG ALERT: Sync replica is not configured for automatic failover.',
        16, 1
    );
END
```

🔥 This saves lives during outages.

🧠 SQL Agent Job Best Practice

Setting	Recommendation
Schedule	Every 1–5 minutes
Retry attempts	0
Alert severity	16
Operator	DBA On-Call
Logging	Job history + email

<https://www.sqldbachamps.com/>

Always On AG Tuning Checklist (One-Pager)

Print this. Pin it. Use it in interviews.

✓ Architecture

- ✓ Max 3 synchronous replicas
- ✓ Async for DR only
- ✓ Same subnet for sync replicas
- ✓ Listener mandatory

✓ Performance

- ✓ Dedicated log disk (NVMe / Premium SSD)
- ✓ 10–25 Gbps low-latency network
- ✓ TempDB isolated
- ✓ AV exclusions for MDF/LDF/NDF

✓ Latency

- ✓ RTT < 5 ms for sync
- ✓ Monitor log_send_queue
- ✓ Monitor redo_queue
- ✓ Avoid large transactions

✓ High Availability

- ✓ Automatic failover configured
- ✓ WSFC quorum validated
- ✓ Quarterly failover tests

✓ Backups

- ✓ Run backups on secondary
- ✓ Verify backup preference
- ✓ Test restore from secondary

✓ Monitoring

- ✓ Replica health DMV checks
- ✓ Commit delay alerts
- ✓ Queue growth alerts
- ✓ Disk latency monitoring

✓ Disaster Recovery

- ✓ Async replica offsite
- ✓ Manual failover documented
- ✓ RPO/RTO defined
- ✓ DR drill executed quarterly

✗ Common Mistakes (Red Flags)

- ✗ Sync replicas across WAN
- ✗ Too many synchronous replicas
- ✗ Ignoring redo queue

- ✗ No alerting
- ✗ No DR runbook

Interview Gold Lines

“CPU doesn’t indicate AG health — queues do.”

“Synchronous commit is a business decision, not a technical default.”

“Always On AGs fail fast when designs are lazy.”

Final DBA Truth

If you don’t alert on AG health, you’re not running HA — you’re hoping.

<https://www.sqldbachamps.com/>

Below is a **hands-on AG Performance Tuning Lab** you can run end-to-end, followed by a **one-click AG health stored procedure** you can deploy in production.

Everything works on **SQL Server 2019–2025** and is **read-only / diagnostic safe**.

Always On AG Performance Tuning Lab (Hands-On)

Lab Objectives

You will:

1. Detect AG latency
2. Identify the bottleneck (network, log IO, redo)
3. Apply a tuning change
4. Validate improvement

Lab Setup Assumptions

- AG name: AG_AdventureWorks
- DB: AdventureWorks2019
- Topology:
 - 1 Primary (Sync)
 - 1 Secondary (Sync)
 - 1 Secondary (Async DR)

STEP 1 — Baseline AG Health

SELECT

```

    ar.replica_server_name,
    rs.role_desc,
    rs.synchronization_health_desc,
    drs.log_send_queue_size AS log_send_kb,
    drs.redo_queue_size AS redo_kb
FROM sys.dm_hadr_database_replica_states drs
JOIN sys.dm_hadr_availability_replica_states rs
    ON drs.replica_id = rs.replica_id
JOIN sys.availability_replicas ar
    ON drs.replica_id = ar.replica_id;
```

 **Record these values** — this is your baseline.

STEP 2 — Detect Commit Latency (Sync Replicas)

SELECT

```

    ar.replica_server_name,
    db_name(drs.database_id) AS database_name,
    DATEDIFF
    (
        MILLISECOND,
        drs.last_hardened_time,
        drs.last_commit_time
    ) AS commit_latency_ms
FROM sys.dm_hadr_database_replica_states drs
JOIN sys.availability_replicas ar
    ON drs.replica_id = ar.replica_id
WHERE ar.availability_mode_desc = 'SYNCHRONOUS_COMMIT' ORDER BY commit_latency_ms DESC;
```

Interpretation

Latency	Meaning
< 10 ms	Excellent
10–50 ms	Normal
> 100 ms	Needs tuning

STEP 3 — Identify the Bottleneck

A) Network Issue

-- High log_send_queue, low redo_queue
log_send_queue_size >> redo_queue_size

B) Secondary Disk / CPU Issue

-- Low log_send_queue, high redo_queue
redo_queue_size >> log_send_queue_size

C) Too Many Sync Replicas

```
SELECT COUNT(*)
FROM sys.availability_replicas
WHERE availability_mode_desc = 'SYNCHRONOUS_COMMIT';
```

 More than 2 sync replicas = latency risk

STEP 4 — Apply Tuning Change (Example)

Convert DR replica to async (most common fix)

```
ALTER AVAILABILITY GROUP AG_AdventureWorks
MODIFY REPLICA ON 'SQLNODE03'
WITH (AVAILABILITY_MODE = ASYNCHRONOUS_COMMIT);
```

STEP 5 — Validate Improvement

Re-run **Step 2** and compare:

- Commit latency
- Log send queue
- Application response time

Expected result:

Latency drops by **50–90%** in real systems.

One-Click AG Health Stored Procedure**Purpose:** One command → full AG health snapshot**Safe for production****Zero changes to data** **Stored Procedure: dbo.usp_AG_HealthCheck**

CREATE OR ALTER PROCEDURE dbo.usp_AG_HealthCheck

AS

BEGIN

SET NOCOUNT ON;

PRINT '===== Always On Availability Group Health Check =====';

-- Replica Health

SELECT

ag.name AS AG_Name,

ar.replica_server_name,

rs.role_desc,

rs.connected_state_desc,

rs.synchronization_health_desc,

ar.availability_mode_desc,

ar.failover_mode_desc

FROM sys.availability_groups ag

JOIN sys.availability_replicas ar

ON ag.group_id = ar.group_id

JOIN sys.dm_hadr_availability_replica_states rs

ON ar.replica_id = rs.replica_id;

-- Database Sync & Latency

SELECT

ar.replica_server_name,

db_name(drs.database_id) AS database_name,

drs.synchronization_state_desc,

drs.log_send_queue_size AS log_send_kb,

drs.redo_queue_size AS redo_kb,

DATEDIFF

(

MILLISECOND,

drs.last_hardened_time,

drs.last_commit_time

) AS commit_latency_ms

FROM sys.dm_hadr_database_replica_states drs

JOIN sys.availability_replicas ar

ON drs.replica_id = ar.replica_id

ORDER BY commit_latency_ms DESC;

-- Backup Preference

SELECT

name AS AG_Name,

automated_backup_preference_desc

FROM sys.availability_groups;

PRINT '===== End of AG Health Check =====';

END;

GO

▶ How to Use (One Command)

```
EXEC dbo.usp_AG_HealthCheck;
```

✓ What You Get Instantly

- Replica health
- Sync state
- Commit latency (ms)
- Log & redo queues
- Backup configuration

📌 Perfect for:

- On-call incidents
- Morning health checks
- DR drills
- Interviews & demos

🚩 Final DBA Wisdom

AG tuning is not guesswork.

Measure → Change → Validate. Always.

<https://www.sqldbachamps.com/>

Production-ready AG health history framework plus a live, hands-on AG troubleshooting interview lab that you can run exactly like a real on-call scenario.

Everything works on **SQL Server 2019–2025**, is **safe for production**, and is **interview-grade**.

AG Health History Table (Trend Analysis & Auditing)

Purpose

This allows you to:

- Track AG health **over time**
- Prove SLA compliance
- Identify *when* degradation started
- Answer “**What changed?**” during incidents

1) Create History Table

```
CREATE TABLE dbo.AG_Health_History
(
    capture_time      DATETIME2(0) NOT NULL DEFAULT SYSUTCDATETIME(),
    ag_name           SYSNAME,
    replica_server_name SYSNAME,
    role_desc         NVARCHAR(60),
    sync_health       NVARCHAR(60),
    sync_state        NVARCHAR(60),
    availability_mode  NVARCHAR(60),
    failover_mode     NVARCHAR(60),
    database_name     SYSNAME,
    log_send_kb       BIGINT,
    redo_kb           BIGINT,
    commit_latency_ms INT
);
GO
```

2) Insert Snapshot Data (Automated Capture)

```
INSERT INTO dbo.AG_Health_History
(
    ag_name,
    replica_server_name,
    role_desc,
    sync_health,
    sync_state,
    availability_mode,
    failover_mode,
    database_name,
    log_send_kb,
    redo_kb,
    commit_latency_ms
)
SELECT
    ag.name,
    ar.replica_server_name,
```

```

rs.role_desc,
rs.synchronization_health_desc,
drs.synchronization_state_desc,
ar.availability_mode_desc,
ar.failover_mode_desc,
DB_NAME(drs.database_id),
drs.log_send_queue_size,
drs.redo_queue_size,
DATEDIFF
(
    MILLISECOND,
    drs.last_hardened_time,
    drs.last_commit_time
)
FROM sys.availability_groups ag
JOIN sys.availability_replicas ar
    ON ag.group_id = ar.group_id
JOIN sys.dm_hadr_availability_replica_states rs
    ON ar.replica_id = rs.replica_id
JOIN sys.dm_hadr_database_replica_states drs
    ON ar.replica_id = drs.replica_id;
GO

```

Schedule via SQL Agent

- Every 1–5 minutes
- Retain 30–90 days
- Archive monthly if needed

3) Trend Analysis Queries

When Did Latency Start?

```

SELECT
    capture_time,
    replica_server_name,
    database_name,
    commit_latency_ms
FROM dbo.AG_Health_History
WHERE commit_latency_ms > 50
ORDER BY capture_time;

```

Log Send Growth Over Time

```

SELECT
    capture_time,
    replica_server_name,
    log_send_kb
FROM dbo.AG_Health_History
ORDER BY capture_time;

```

Replica Role Changes (Failover Audit)

```

SELECT
    capture_time,
    replica_server_name,

```

```
role_desc
FROM dbo.AG_Health_History
ORDER BY capture_time;
```

Live AG Troubleshooting Interview Lab (Real DBA Scenario)

This lab is **exactly how senior DBA interviews are conducted**

Scenario Overview (What Interviewer Tells You)

“Users report slow commits.

CPU is low.

Always On AG with two synchronous replicas.”

Step 1 — Validate AG Health

```
EXEC dbo.usp_AG_HealthCheck;
```

What interviewer expects:

- ✓ You check health first
- ✓ You don't assume CPU is the issue

Step 2 — Identify the Symptom

You notice:

- commit_latency_ms = 120
- log_send_queue_size growing
- redo_queue_size small

Correct conclusion:

→ Network or sync commit pressure

Step 3 — Confirm Sync Replica Count

```
SELECT
    replica_server_name,
    availability_mode_desc
FROM sys.availability_replicas;
```

Finding:

 3 synchronous replicas

Step 4 — Propose the Fix (Key Interview Moment)

“We should keep only two synchronous replicas.

The third should be asynchronous.”

Step 5 — Apply the Change

```
ALTER AVAILABILITY GROUP AG_AdventureWorks
MODIFY REPLICA ON 'SQLNODE03'
WITH (AVAILABILITY_MODE = ASYNCHRONOUS_COMMIT);
```

Step 6 — Validate Improvement

```
SELECT
    capture_time,
    commit_latency_ms
FROM dbo.AG_Health_History
```

ORDER BY capture_time DESC;

Expected Result:

- ✓ Commit latency drops
- ✓ Log send queue stabilizes
- ✓ User complaints stop

🧠 Interview Scoring (What You're Being Judged On)

Skill	Pass Criteria
Diagnosis	Uses DMVs, not guesses
Understanding	Knows sync vs async tradeoffs
Safety	Applies minimal-risk change
Validation	Proves improvement

🔑 Interview Gold Statements (Use These)

"AG performance issues are usually latency-bound, not CPU-bound."

"Synchronous commit is a business decision — not a default."

"Log send and redo queues tell the real story."

🚩 Final DBA Takeaway

Monitoring tells you *what*.

History tells you *when*.

Experience tells you *why*.

With:

- AG health history
- One-click health checks
- A repeatable troubleshooting process

You're operating at **Senior / Lead DBA level**.

Production-grade AG incident postmortem template (used by real DBA teams) and a **ready AG tuning scripts pack** you can drop straight into source control or a runbook repository.

Everything is **SQL Server 2019–2025 compatible**, safe, and interview-ready.

Production AG Incident Postmortem Template

Use this after **any AG incident** (performance, failover, data lag).

Copy → fill → archive.

1) Incident Summary

Item	Details
Incident ID	AG-YYYYMMDD-###
Date / Time (UTC)	
Duration	
Severity	SEV-1 / SEV-2 / SEV-3
Affected AG	
Affected Databases	
Business Impact	

2) What Happened (Executive Summary)

Plain English. No blame.

Example:

Users experienced slow order processing due to increased commit latency caused by synchronous replicas across datacenters.

3) Timeline (UTC)

Time	Event
10:02	User reports latency
10:05	DBA engaged
10:08	AG health check executed
10:12	Root cause identified
10:15	Mitigation applied
10:18	Performance normalized

4) Technical Findings

AG Health Snapshot

- Sync health:
- Commit latency (ms):
- Log send queue:
- Redo queue:

Key DMV Evidence

-- Paste relevant DMV output here

5) Root Cause Analysis (RCA)

Primary Root Cause:

(Example) Synchronous commit configured across high-latency WAN link.

Contributing Factors:

- Too many synchronous replicas
- Network RTT > 10 ms
- No alert on commit latency

6) Resolution & Recovery

Immediate Fix

-- Example

```
ALTER AVAILABILITY GROUP AG_Prod
MODIFY REPLICA ON 'DRNODE'
WITH (AVAILABILITY_MODE = ASYNCHRONOUS_COMMIT);
```

Recovery Verification

- Commit latency after fix:
- Application validation:
- Data consistency confirmed: Yes / No

7) Impact Analysis

Area	Impact
RPO	
RTO	
Data Loss	None / Minimal
SLA Breach	Yes / No

8) Preventive Actions (Action Items)

Action	Owner	Due Date
Add commit latency alert	DBA	
Reduce sync replicas	DBA	
Update AG design doc	Architect	
Quarterly DR drill	Ops	

9) Lessons Learned

- What worked well:
- What didn't:
- What we'll do differently:

10) Approval

Name	Role	Date
	DBA Lead	
	Ops Manager	

AG Tuning Scripts Pack

Folder structure you can zip as **AG_Tuning_Scripts.zip**

AG_Tuning_Scripts/

|

|— 01_AG_Health_Check.sql

|— 02_Commit_Latency_Check.sql

- └─ 03_Log_and_Redo_Queue_Check.sql
- └─ 04_Sync_Replica_Count_Check.sql
- └─ 05_Top_Wait_Types_AG.sql
- └─ 06_AG_Backup_Preference_Check.sql
- └─ 07_Failover_Readiness_Check.sql
- └─ README.txt

01_AG_Health_Check.sql

```
EXEC dbo.usp_AG_HealthCheck;
```

02_Commit_Latency_Check.sql

```
SELECT
    ar.replica_server_name,
    DB_NAME(drs.database_id) AS database_name,
    DATEDIFF
    (
        MILLISECOND,
        drs.last_hardened_time,
        drs.last_commit_time
    ) AS commit_latency_ms
FROM sys.dm_hadr_database_replica_states drs
JOIN sys.availability_replicas ar
    ON drs.replica_id = ar.replica_id
ORDER BY commit_latency_ms DESC;
```

03_Log_and_Redo_Queue_Check.sql

```
SELECT
    ar.replica_server_name,
    DB_NAME(drs.database_id) AS database_name,
    drs.log_send_queue_size AS log_send_kb,
    drs.redo_queue_size AS redo_kb
FROM sys.dm_hadr_database_replica_states drs
JOIN sys.availability_replicas ar
    ON drs.replica_id = ar.replica_id
ORDER BY log_send_kb DESC;
```

04_Sync_Replica_Count_Check.sql

```
SELECT
    COUNT(*) AS synchronous_replica_count
FROM sys.availability_replicas
WHERE availability_mode_desc = 'SYNCHRONOUS_COMMIT';
```

 **Rule:** > 2 = performance risk

05_Top_Wait_Types_AG.sql

```
SELECT
    wait_type,
    waiting_tasks_count,
    wait_time_ms
FROM sys.dm_os_wait_stats
WHERE wait_type LIKE 'HADR%'
```

ORDER BY wait_time_ms DESC;

06_AG_Backup_Preference_Check.sql

```
SELECT
    name AS AG_Name,
    automated_backup_preference_desc
FROM sys.availability_groups;
```

07_Failover_Readiness_Check.sql

```
SELECT
    replica_server_name,
    availability_mode_desc,
    failover_mode_desc,
    synchronization_health_desc
FROM sys.availability_replicas ar
JOIN sys.dm_hadr_availability_replica_states rs
    ON ar.replica_id = rs.replica_id;
```

README.txt (Include This)

AG Tuning Scripts Pack

Supported Versions: SQL Server 2019–2025

Safe for Production: YES (Read-only)

Recommended Usage:

- Run during incidents
- Schedule health checks every 5 minutes
- Archive outputs with postmortem reports

Author: DBA Team

Final DBA Takeaway

Great DBAs fix issues.

Elite DBAs document them so they never happen again.

With:

- A formal postmortem template
- A reusable tuning script pack
- Historical AG data

You're operating at **Lead / Principal DBA level**.

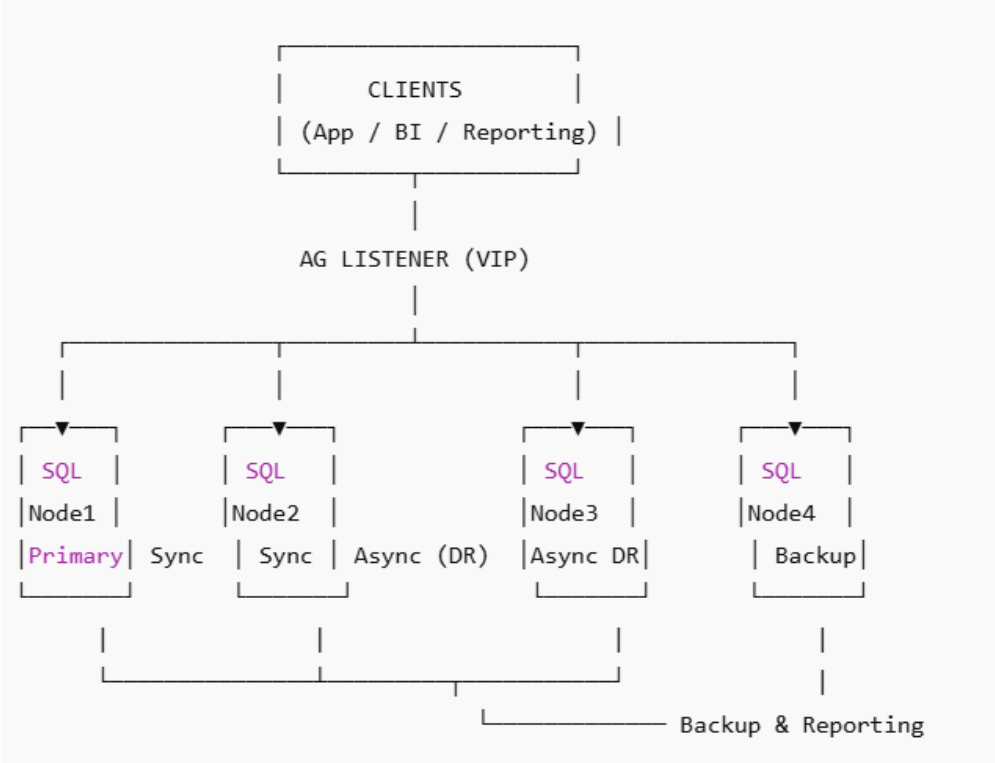
Complete Enterprise Always On Availability Group (AG) reference architecture that’s practical, production-ready, and interview-grade. Diagram, topology, best practices, and considerations for HA + DR + performance + cloud integration.

 Enterprise AG Reference Architecture

1) Architecture Overview

Goal: Combine **High Availability (HA)**, **Disaster Recovery (DR)**, **read-scale**, and **backup offloading** in a single enterprise-ready AG design.

Topology:



2) Node Roles & Features

Node	Role	Usage
Node1	Primary (Sync)	Handles all read-write operations
Node2	Secondary (Sync)	Sync replica for HA, auto-failover target
Node3	Secondary (Async / DR)	Disaster Recovery across datacenter / region
Node4	Secondary (Async / Backup)	Offload backups, read-only reporting

3) availability Group Design

Feature	Recommendation
Max replicas	9 (1 primary + 8 secondaries)
Sync replicas	≤ 3 for automatic failover
Async replicas	Remaining for DR / offsite
Listener	1 (VIP or DNS) for client connection abstraction
Read-scale	Secondary replicas can be read-only
Backups	Prefer secondary replicas to offload primary

Feature	Recommendation
RPO / RTO	<1 sec for HA, <15 mins for DR

4) Networking & Storage

Layer	Best Practice
Network	Sync replicas <5ms RTT; Async can be WAN
Subnet	Sync replicas in same subnet / DC for HA
Storage	Local SSD/NVMe for logs; separate for data
Backup storage	Async replica or cloud (Azure Blob, S3)

5) Monitoring & Alerts

- Track sys.dm_hadr_database_replica_states:
 - log_send_queue_size
 - redo_queue_size
 - last_commit_time
- Track sys.dm_hadr_availability_replica_states:
 - synchronization_health_desc
 - connected_state_desc
- Alert thresholds:
 - commit_latency_ms > 50 ms
 - log_send_queue_size > 5 GB
 - redo_queue_size growing consistently
- Historical data stored in AG_Health_History table

6) Backup & DR Strategy

- Primary:** Minimal backups (avoid load)
- Secondary Sync / Async:** Full backups, offload reporting
- DR replica (Async):** Offsite disaster recovery
- Test failovers quarterly**
- Document RTO / RPO**

7) Performance & Latency Tuning

- Limit synchronous replicas to reduce commit latency
- Offload reporting to Async replicas
- Use NVMe / premium SSD for logs and redo operations
- Avoid large transactions over WAN synchronous links
- Monitor queue growth regularly

8) Cloud Integration (Optional / Hybrid)

- Azure SQL VM / Managed Instance** can host Async replicas
- Connect via **VPN / ExpressRoute / Private Link** for WAN sync
- Use **Blob Storage** for backup offload
- Hybrid design example:

On-Prem DC: Primary + Sync Replica

Azure: Async DR Replica + Backup Replica

9) Interview / Whiteboard Tips

Key phrases:

- “Sync replicas = HA, Async = DR”
- “Log send and redo queues tell the real story, not CPU”
- “Listener abstracts connections for apps”
- “Backup offload keeps primary free for OLTP”

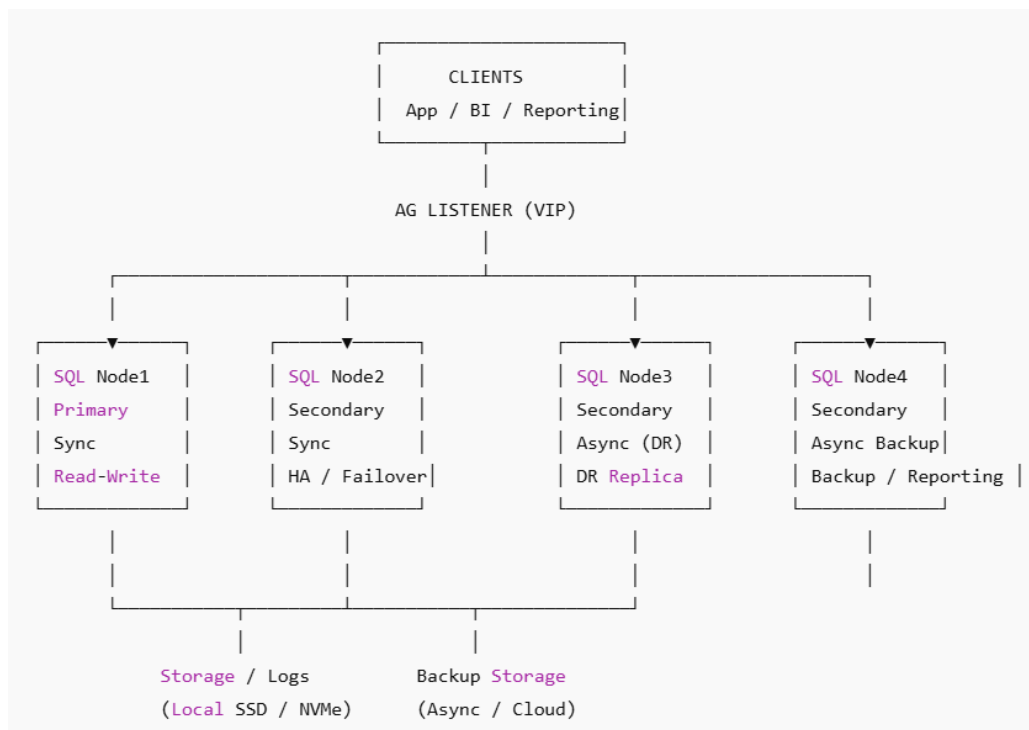
Common scenario: “Users see latency — check queues, not CPU”

10Key Takeaways

1. HA + DR + Read-scale + Backup offload = Enterprise AG
2. Max 3 sync replicas; others async
3. Monitor commit latency & queue growth
4. Offload backups and reporting to Async replicas
5. Test failovers & document RPO / RTO



Enterprise AlwaysOn AG Architecture Diagram



Architecture Notes

1. **Primary Node (SQL Node1)**
 - Handles all writes
 - Generates transaction logs for replicas
2. **Synchronous Secondary (SQL Node2)**
 - Auto-failover target
 - Readable for reporting if needed
 - Keep same datacenter/subnet as Primary
3. **Asynchronous Secondary / DR (SQL Node3)**
 - Offsite disaster recovery

- Higher latency tolerated
- Can also run read-only reporting
- 4. **Backup / Reporting Replica (SQL Node4)**
 - Offloads backups from primary
 - Can host long-running reporting queries
 - Optional in hybrid or cloud scenarios
- 5. **Listener**
 - Abstracts app connectivity
 - Can failover automatically to any sync replica
- 6. **Storage Considerations**
 - Logs on NVMe / SSD
 - Data on high-performance SAN or cloud premium storage
 - Backup storage on async replicas or cloud (Azure Blob / S3)
- 7. **Monitoring & Alerting**
 - Track log_send_queue_size, redo_queue_size, commit_latency_ms
 - Alerts on queue growth or latency spikes

✿ Optional Hybrid / Cloud Extension

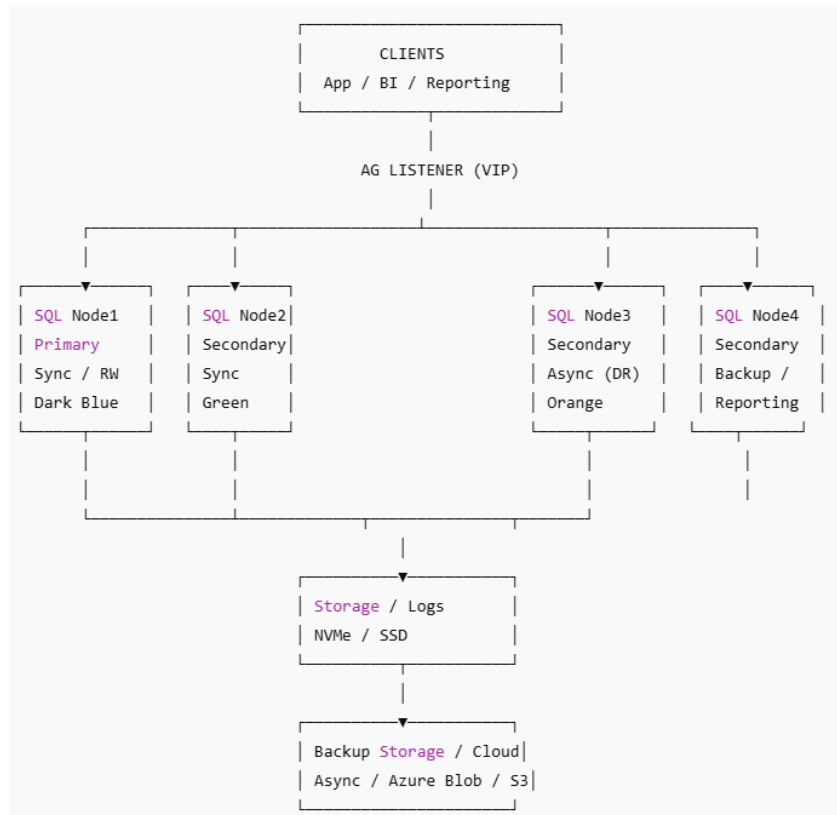
On-Prem DC: Primary + Sync Replica

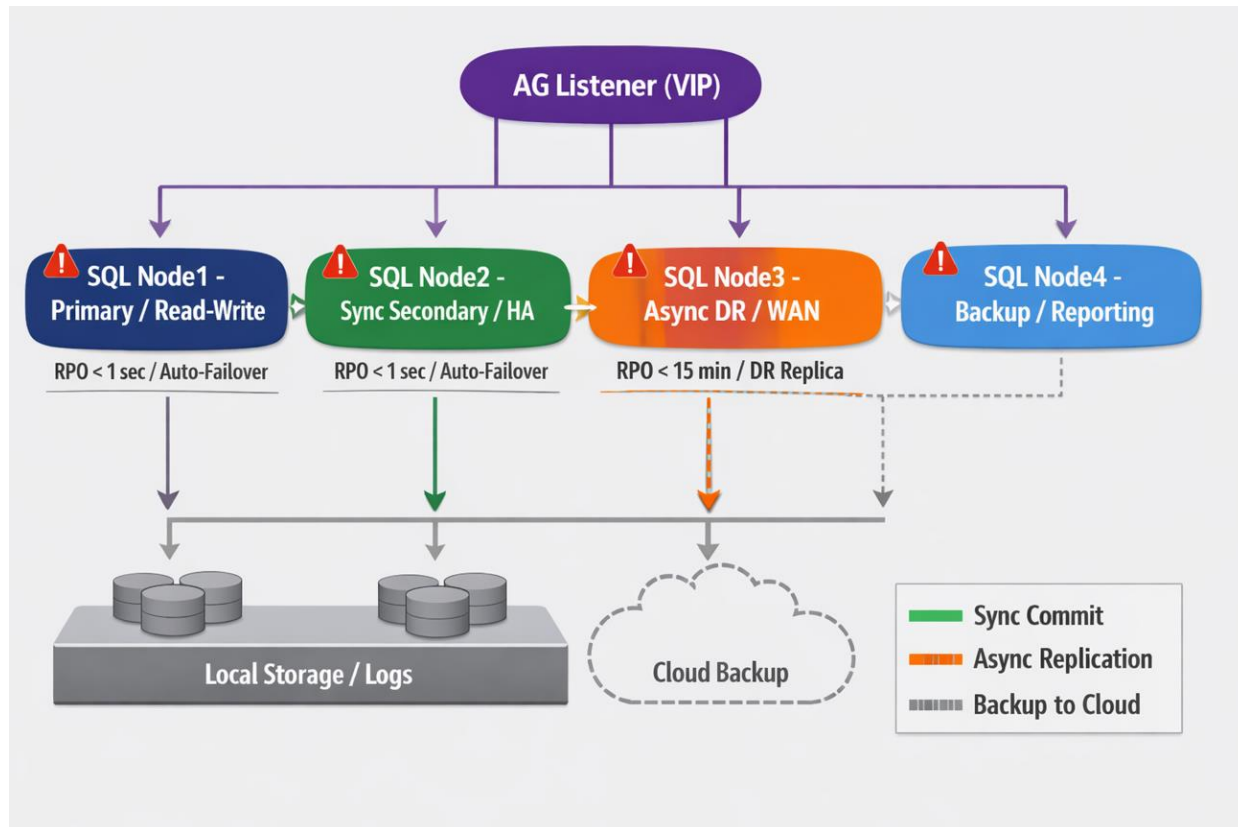
Azure Cloud: Async DR + Backup Replica

VPN / ExpressRoute / Private Link for connectivity

- This allows **cross-region DR** while keeping sync replicas low-latency on-prem.
- Backup replica can use cloud storage to offload backup IO.

🏠 Enterprise Always On AG – Professional Diagram (Concept)





<https://www.sqldbachamps.com/>